

AI ENABLEMENT | CLAUDE CODE KICKOFF

# Scenariusz prowadzącego

Polskie notatki do 53 angielskich slajdów

|        |                      |
|--------|----------------------|
| FORMAT | A4   wersja do druku |
| CZAS   | około 2 godzin       |
| TRYB   | materiał modułowy    |

# Jak korzystać z dokumentu

Scenariusz dotyczy angielskiej prezentacji **index.html**. Tekst poniżej jest przeznaczony dla prowadzącego. Nie czytaj go słowo w słowo. Traktuj go jako mapę argumentów, przykładów i przejść. Całość daje około dwóch godzin mówienia, ale sekcje można skracać.

|       |              |                                           |
|-------|--------------|-------------------------------------------|
| START | Slajdy 1-4   | Cel i mapa spotkania                      |
| 01    | Slajdy 5-12  | Poziomy adopcji i spec-driven development |
| 02    | Slajdy 13-19 | Architektura systemów agentowych          |
| 03    | Slajdy 20-26 | Claude Code                               |
| 04    | Slajdy 27-32 | Niezawodność i guardrails                 |
| 05    | Slajdy 33-37 | SDLC i współpraca                         |
| 06    | Slajdy 38-47 | Koszt, jakość i analiza porażek           |
| 07    | Slajdy 48-53 | Hackathon i zamknięcie                    |

## 01

## AI Enablement

**Cel:** ustawić kontekst spotkania. Powiedz, że to kickoff do hackathonu, a nie kurs obsługi kolejnego edytora. Głównym tematem jest **kontrolowana autonomia**: ile pracy możemy oddać Claude Code, jakie granice muszą pozostać twarde i jak rozpoznać, że wynik jest dobry.

Podkreśl, że będziemy mówić o **sposobie pracy zespołu**, a nie o magicznych promptach. Uczestnicy mają po spotkaniu umieć świadomie wybrać poziom autonomii dla swojego problemu.

**Przejdźcie:** „Zacznijmy od krótkiego reality checku: czym właściwie różni się model od systemu agentowego?”

## 02

## Reality check

**Cel:** oddzielić model od całego systemu. Model generuje następną odpowiedź, natomiast agent działa w pętli: obserwuje środowisko, wybiera narzędzie, wykonuje krok i analizuje wynik.

Wyjaśnij, że jakość nie zależy wyłącznie od „inteligencji modelu”. Równie ważne są **kontekst, narzędzia, uprawnienia**, środowisko wykonania oraz mechanizm sprawdzający wynik. Ten sam model może działać znakomicie w dobrze zaprojektowanym workflow i fatalnie w chaotycznym.

**Akcent:** kiedy agent ma dostęp do terminala, błędna odpowiedź może stać się błędnym działaniem.

## 03

## Kickoff goal

Przedstaw cztery oczekiwane rezultaty spotkania. Po pierwsze, zespół ma umieć nazwać **poziom autonomii**. Po drugie, ma wiedzieć, gdzie ustawić **granice wykonawcze**. Po trzecie, ma mierzyć **wynik razem z kosztem**. Po czwarte, ma analizować porażkę na podstawie śladów, nie intuicji.

Zaznacz, że prezentacja nie rozstrzygnie każdego detalu technologicznego. Da wspólny język do podejmowania decyzji podczas hackathonu.

## 04

## Session map

Krótko omów siedem bloków. Powiedz uczestnikom, że pierwsze trzy dają model mentalny, kolejne trzy dotyczą bezpiecznego skalowania, a ostatni przekłada zasady na hackathon.

Warto uprzedzić, że część slajdów jest celowo „na zapas”. Jeśli dyskusja pójdzie głębiej w architekturę, można skrócić sekcję narzędziową. **Deck jest modułowy.**

## 05

## Adoption levels

**Cel:** zmienić sposób myślenia o dojrzałości. Powiedz, że zakup Claude Code nie przenosi organizacji automatycznie na wyższy poziom. Dojrzałość opisuje **powtarzalny proces**, a nie licencja ani liczba wygenerowanych linii.

Wyjaśnij też, że poziom oceniamy **dla konkretnego procesu**. Zespół może mieć wysoką autonomię w generowaniu testów i bardzo niską w zmianach IAM.

## 06

## Five adoption levels

Przejdź przez pięć poziomów od lewej do prawej. Na poziomie pierwszym delegujemy odpowiedź. Na drugim małą edycję. Na trzecim całe zadanie. Na czwartym agent realizuje kontrakt zapisany w specyfikacji. Na piątym obsługuje część procesu w wyznaczonych granicach.

Podkreśl, że jest to **model warsztatowy**, a nie branżowy standard. Jego wartość polega na rozmowie o zakresie decyzji i wymaganych zabezpieczeniach.

## 07

## Levels 1 and 2

Wyjaśnij, że na dwóch pierwszych poziomach człowiek nadal steruje zmianą krok po kroku. Claude pomaga z wyjaśnieniem, fragmentem kodu lub niewielką edycją, lecz człowiek wybiera kontekst i ocenia każdy rezultat.

To dobry poziom dla nowych użytkowników oraz obszarów o wysokim ryzyku. **Niska autonomia nie oznacza niskiej wartości**. Czasem kilka trafnych wyjaśnień oszczędza więcej czasu niż duża automatyczna zmiana.

## 08

## Level 3

Tutaj pojawia się właściwy **agentic coding**. Delegujemy wynik: agent bada repo, tworzy plan, zmienia kilka plików, uruchamia testy i reaguje na ich rezultat.

Zwróć uwagę, że kontrola nie znika. Przenosi się z każdej linii kodu na **plan, uprawnienia, testy** i końcowy **diff**. To kluczowa zmiana roli inżyniera: mniej pisania, więcej formułowania celu oraz oceny dowodów.

## 09

## Level 4

Powiedz, że wyższa autonomia wymaga lepszego kontraktu. Specyfikacja zawiera problem, kryteria akceptacji, ograniczenia i sposób weryfikacji. Agent nie musi zgadywać decyzji produktowych.

Wiedza zespołu powinna żyć w repozytorium jako **CLAUDE.md**, rules, skills, hooks, dokumentacja i testy. Dzięki temu jakość nie zależy od pamięci jednej osoby ani od treści poprzedniej rozmowy.

## 10

## Level 5

Opisz ten poziom ostrożnie. Agent może obsługiwać kolejkę zadań albo fragment pipeline, ale działa w **sandboxie**, z budżetem i warunkiem zatrzymania. To nie jest zgoda na nieograniczone działanie.

Powiedz wprost, że poziom piąty ma sens dopiero przy stabilnym procesie, obserwowalności i dobrych testach. Automatyzowanie chaosu produkuje chaos szybciej i drożej.

## 11

## Moving up the curve

Omów cztery bramki. **Powtarzalność** mówi, czy zadanie ma znany wzorzec. **Weryfikacja** mówi, czy potrafimy odróżnić sukces od pozoru sukcesu. **Izolacja** ogranicza promień rażenia. **Obserwowalność** pozwala odtworzyć decyzje i koszt.

Zaznacz kolejność: najpierw standaryzacja, potem automatyzacja, dopiero później autonomia. Jeśli bramka nie jest spełniona, cofamy poziom delegacji.

## 12

## Spec-driven frameworks

**Cel:** pokazać, że **spec-driven development** jest wzorcem pracy, a nie jednym produktem ani obowiązkową metodyką. Wspólny mianownik tych podejść to zapisanie intencji przed implementacją, przekazywanie kontekstu przez trwałe artefakty oraz osobny krok sprawdzający zgodność wyniku ze specyfikacją.

**GitHub Spec Kit** proponuje czytelny łańcuch: specyfiky, plan, tasks, implement i converge. Każdy etap wytwarza artefakt Markdown dla kolejnego etapu. Podkreśl, że implementacja nie zaczyna się od luźnego promptu, tylko od uzgodnionego opisu problemu i kryteriów.

**GSD Core** organizuje pracę w pętlę discuss, plan, execute, verify i ship. Jego charakterystyczny element to przenoszenie decyzji przez pliki stanu i uruchamianie ciężkiej pracy w świeżych kontekstach. Dzięki temu framework próbuje ograniczać **context rot**, czyli pogorszenie jakości wraz z przepełnianiem okna kontekstowego.

**BMad Method** dobiera głębokość procesu do wielkości zmiany. Prosta poprawka może przejść bez rozbudowanego planowania, a większa inicjatywa otrzymuje dodatkowe perspektywy produktowe, architektoniczne, implementacyjne i testowe. Najważniejsza jest tu zasada **right-sized process**: proces powinien być wystarczający, ale nie cięższy niż problem.

Nie przedstawiaj tych frameworków jako rankingu ani kolejnych poziomów dojrzałości. Jako wskazówkę dla zespołu powiedz: wybieramy **najlżejszy proces**, który zachowuje decyzje, kryteria akceptacji i dowody weryfikacji. Nazwa narzędzia jest mniej ważna niż jakość kontraktu i dyscyplina przejścia od specyfikacji do testu.

**Przejście:** „Framework porządkuje workflow. Teraz zobaczmy, jaki system faktycznie wykonuje tę pracę i gdzie powstają decyzje agenta.”

## 13 Agentic system architecture

To otwarcie sekcji architektonicznej. Powiedz, że agent nie jest osobnym rodzajem modelu. To model osadzony w **pętli ze sprzężeniem zwrotnym**, który może korzystać z narzędzi i stanu środowiska.

Ustaw pytanie przewodnie: „Co dokładnie znajduje się między intencją użytkownika a skutkiem w systemie?”

## 14 Minimal agent architecture

Przejdź po elementach diagramu. **Cel i kontrakt** mówią, czego chcemy. **Model z orchestrator-em** wybiera kolejne działania. **Narzędzia** wykonują pracę. **Środowisko** zwraca obserwacje. **Weryfikator** ocenia postęp.

Wyjaśnij, że słaby interfejs narzędzia może zepsuć pracę dobrego modelu. Nazwy parametrów, komunikaty błędów i zakres uprawnień są częścią architektury agenta.

## 15 The agent loop

Omów sześć kroków jako powtarzalny cykl. Najważniejszy jest krok czwarty: agent musi **odczytać prawdziwy wynik** działania. Bez tego działa na podstawie własnych założeń.

Podaj przykład: po edycji kodu agent uruchamia test, czyta błąd, poprawia implementację i ponawia test. Test stanowi ground truth. Pewnie brzmiący opis modelu nim nie jest.

## 16 Workflow and agent

Porównaj deterministyczny workflow z agentem. Workflow ma z góry ustaloną sekwencję, więc łatwiej przewidzieć koszt i miejsce awarii. Agent dynamicznie wybiera drogę, więc radzi sobie z otwartymi problemami, lecz wymaga silniejszych limitów.

**Reguła praktyczna:** jeśli ścieżkę można opisać wcześniej, preferuj workflow. Agent ma sens wtedy, gdy liczba i rodzaj kroków zależą od sytuacji.

## 17 Architecture patterns

Krótko scharakteryzuj pięć wzorców. Chaining dzieli problem na stałe kroki. Routing wybiera wyspecjalizowaną ścieżkę. Parallelization dzieli niezależną pracę. Orchestrator dynamicznie tworzy podzadania. Evaluator ocenia wynik i uruchamia kolejną iterację.

Nie przedstawiaj ich jako poziomów dojrzałości. To **narzędzia projektowe**. Dodaj złożoność tylko wtedy, gdy poprawia mierzony wynik.

## 18 Autonomy matrix

Pokaż, że autonomia ma co najmniej cztery wymiary. Agent może sam planować, lecz mieć wyłącznie odczyt. Może edytować pliki, ale tylko w lokalnym sandboxie. Może działać długo, ale z niskim budżetem.

Zachęć uczestników, aby zamiast pytania „czy agent jest autonomiczny?” pytali osobno o **decyzje, narzędzia, środowisko i czas działania**.

## 19 Risk budget

Wyjaśnij osie: zasięg skutków i trudność odwrócenia. Czytanie repo oraz lokalne analizy mają niski koszt błędu. Edycja i testy powinny działać w sandboxie. Sieć, publikacja i sekrety wymagają zgody. Produkcja i nieodwracalne działania pozostają poza agentem lub wymagają osobnego procesu.

Podkreśl **promień rażenia**. O poziomie kontroli decyduje skutek błędu, nie atrakcyjność dema.

## 20 Claude Code

Otwórz sekcję narzędziową. Claude Code łączy model z repozytorium, terminalem i zestawem rozszerzeń. Nie omawiamy każdej funkcji produktu, tylko warstwy, które wpływają na niezawodność i pracę zespołu.

## 21 Claude Code mental model

Omów role warstw. **CLAUDE.md i rules** dostarczają kontekst. **Skills** opisują wiedzę lub procedury na żądanie. **Hooks** automatyzują zdarzenia. **Subagents** izolują pracę, a agent teams koordynują niezależne sesje. **MCP** daje dostęp do świata zewnętrznego.

Akcentuj, że te mechanizmy się uzupełniają. Nie wrzucamy wszystkiego do jednego ogromnego CLAUDE.md.

## 22 Project memory

Wyjaśnij strukturę katalogów. CLAUDE.md zawiera informacje potrzebne zawsze: komendy, konwencje i ogólną architekturę. Rules mogą dotyczyć tylko wybranych ścieżek. Skills oraz agents ładujemy do konkretnych zadań. **.mcp.json** opisuje współdzielone integracje.

Podkreśl **Git**. Instrukcje zespołowe powinny przechodzić review tak samo jak kod.

## 23 Choosing the mechanism

Przejdź po tabeli jako drzewie decyzji. Informacja potrzebna zawsze trafia do CLAUDE.md. Reguła dla części repo trafia do rule. Powtarzalna procedura staje się skillem. Automatyczna, deterministyczna reakcja staje się hookiem. Duże zadanie poboczne może wykonać subagent. Dostęp do systemu zewnętrznego zapewnia MCP.

**Pułapka:** używanie promptu jako substytutu mechanizmu egzekwującego.

## 24 Context

Wyjaśnij, że kontekst jest ograniczonym budżetem uwagi i kosztu. Najwyższy priorytet mają cel i kryteria akceptacji. Potem konkretne pliki oraz działający wzorzec. Następnie reguły dotyczące zadania. Logi, historia i dokumentacja powinny być pobierane na żądanie.

Duży kontekst nie gwarantuje lepszego wyniku. Może rozmyć ważne instrukcje i podnieść koszt każdego następnego kroku.

## 25 Permissions

To jeden z najważniejszych slajdów. Model może zaproponować wywołanie narzędzia, ale runtime ocenia hook, deny, ask i allow. Instrukcja „nie rób X” w CLAUDE.md wpływa na zachowanie, ale nie jest twardą barierą.

Powiedz jasno: **soft guardrail** kieruje modelem, **hard guardrail** ogranicza możliwość wykonania. Dla działań ryzykownych potrzebujemy drugiego rodzaju.

## 26 Practical working rhythm

Przedstaw rekomendowany rytm: najpierw badanie repo, następnie jawny plan, zatwierdzenie granic, implementacja z testami i na końcu review diffu. To daje człowiekowi naturalne punkty kontroli.

Krótko omów komendy. **/context** pokazuje zużycie okna, **/compact** porządkuje historię, **/clear** zaczyna nowy temat, **/permissions** pokazuje reguły, **/usage** koszt, a **/diff** zmianę.

## 27 Reliability

Otwórz sekcję stwierdzeniem: zaufanie do agenta nie wynika z dobrego tonu odpowiedzi. Wynika z **dowodów**, **granic** i powtarzalnego procesu.

Zapowiedz, że halucynacja w codingu przyjmuje kilka form i nie zawsze wygląda jak wymyślony fakt.

## 28 Types of hallucination

Omów trzy typy. **Fałszywy fakt** to nieistniejące API lub opcja. **Fałszywy stan** to założenie o repo lub środowisku bez odczytu. **Fałszywe zakończenie** to deklaracja sukcesu bez testu.

Podaj przykład ostatniego typu: agent pisze „wszystkie testy przechodzą”, choć nie uruchomił komendy. To częsty i kosztowny błąd operacyjny.

## 29 Evidence contract

Pokaż zasadę claim-evidence. Każde twierdzenie ma odpowiadający mu dowód: log testu, aktualną dokumentację, diff albo kryteria akceptacji.

Zachęć zespół do formułowania zadań tak, aby agent musiał pokazać **źródło prawdy**. Gdy dowodu brakuje, oczekujemy jawnej niepewności i propozycji weryfikacji.

## 30 Guardrail stack

Wyjaśnij cztery warstwy od dołu. Kontekst kształtuje zachowanie. Runtime ogranicza działanie. Weryfikacja ocenia wynik. Proces przypisuje ownership, review i rollback.

Żadna warstwa nie wystarcza sama. Dobre instrukcje nie zastąpią sandboxa, a sandbox nie sprawdzi logiki biznesowej. **Defense in depth** oznacza nakładające się zabezpieczenia.

## 31 Boundaries outside context

Pokaż konkretne mapowanie reguły na mechanizm. Sekrety chronimy przez brak dostępu i deny. Destrukcyjne komendy przez deny oraz hook. Publikację przez osobną tożsamość i brak uprawnień. Koszt przez limit czasu, budżetu i iteracji.

To odpowiedź na obawę, że guardrails „zginą w kontekście”. Twarda granica nie zależy od pamięci modelu.

## 32 Prompt injection

Wyjaśnij atak pośredni. Agent czyta issue, README lub stronę, w której ktoś umieścił instrukcję. Model może potraktować ją jako polecenie i użyć narzędzia.

Obrona obejmuje ograniczenie źródeł, wyraźne rozdzielenie danych od instrukcji, minimalne uprawnienia i zgodę na skutki zewnętrzne. **Zaufany connector nie oznacza zaufanej treści**.

## 33 SDLC and collaboration

Otwórz sekcję stwierdzeniem, że agent nie tworzy równoległego SDLC. Pracuje w istniejącym systemie kontroli jakości: repo, CI, review i procesie wydania.

## 34 Agent in the SDLC

Przejdź przez cykl. Problem opisuje wartość i ryzyko. Spec usuwa niejednoznaczność. Plan ujawnia zakres. Zmiana powstaje w izolacji. Dowody obejmują testy, skany i trace. Człowiek wykonuje review i odpowiada za merge.

Podkreśl, że dowody powinny podróżować razem ze zmianą, na przykład w opisie PR.

## 35 Division of responsibility

Wyjaśnij granicę odpowiedzialności. Agent może zaproponować plan, ale człowiek zatwierdza cel i kompromisy. Agent wykonuje oraz testuje, człowiek interweniuje przy dryfie. Agent dostarcza dowody, człowiek ocenia architekturę i ryzyko. Agent przygotowuje release, człowiek autoryzuje zewnętrzny skutek.

**Odpowiedzialność organizacyjna nie przechodzi na model.**

## 36 Several people, one topic

Powiedz, że wspólna specyfikacja musi zdefiniować interfejsy, kryteria oraz decyzje ADR. Dopiero potem dzielimy pracę na rozłączne strumienie, najlepiej w osobnych worktrees i zestawach plików.

Na końcu potrzebujemy właściciela integracji. Równoległość bez ownership zwiększa konflikty i koszt review.

## 37 Subagent and agent team

Porównaj trzy poziomy. Subagent wykonuje zadanie w osobnym kontekście i zwraca podsumowanie. Agent team pozwala niezależnym sesjom komunikować się oraz współdzielić listę zadań. Zespół ludzi podejmuje decyzje i integruje całość.

Zaznacz, że agent teams są **eksperymentalne**. Zacznij od researchu i review, zanim powierzysz równoległą edycję kodu.

## 38 Cost and quality

Otwórz sekcję FinOps. Koszt nie jest jedynie cennikiem modelu. Wynika z architektury pętli, rozmiaru kontekstu, liczby prób i równoległych agentów.

## 39

## Cost anatomy

Omów cztery mnożniki. Większy kontekst podnosi koszt każdego kroku. Droższy model zwiększa stawkę. Długa pętla mnoży liczbę zapytań. Równoległe gałęzie tworzą osobne okna kontekstu.

Najpierw ograniczamy zbędną pracę systemu, a dopiero potem stroimy pojedynczy prompt.

## 40

## FinOps levers

Przejdź po pięciu dźwigniach. Nowa sesja usuwa nieaktualny kontekst. Dobór modelu dopasowuje koszt do trudności. Skills i scoped rules ładują wiedzę na żądanie. Konkretne AC oraz szybkie testy skracają pętlę. **/usage** i telemetry umożliwiają pomiar.

Najlepsza metryka to **koszt na zaakceptowane zadanie**, nie sam koszt sesji.

## 41

## RTK, Caveman, Ponytail

Wyjaśnij, że trzy narzędzia redukują różne rodzaje marnotrawstwa. **RTK** filtruje hałaśliwy output terminala, zanim trafi do kontekstu. **Caveman** skraca prozę odpowiedzi agenta. **Ponytail** stosuje YAGNI i szuka prostszego rozwiązania z mniejszą ilością kodu.

Dodaj zastrzeżenia. RTK nie redukuje całego rachunku. Caveman sam zużywa kontekst i może nie opłacać się dla krótkiej odpowiedzi. Ponytail nie może usuwać kodu potrzebnego do spełnienia AC. **Mierz osobno input, output i ilość kodu.**

## 42

## Quality regression

Wyjaśnij, że pogorszenie jakości nie musi oznaczać „gorszego modelu”. Przyczyną może być nowa wersja modelu, rozrastający się kontekst, zmienione API narzędzia, nowe dane albo osłabiony proces review.

Diagnostujemy cały system. W przeciwnym razie łatwo naprawić niewłaściwą warstwę.

## 43

## Regression protection

Przedstaw pętlę ochronną. Golden set powinien zawierać realne zadania. Każdy przypadek uruchamiamy kilka razy, bo wynik jest probabilistyczny. Oceniamy outcome i trajektorię. Nową konfigurację wdrażamy jako canary, obserwujemy i utrzymujemy możliwość rollbacku.

Bez zapisanej wersji modelu, narzędzi i konfiguracji porównanie jest niewiarygodne.

## 44 Failure analysis

Wyjaśnij różnicę między trace a outcome. Trace pokazuje, co agent robił: prompty, decyzje, tool calls i poprawki. Outcome pokazuje stan końcowy: repo, testy, skutki i koszt.

Diagnoza szuka **pierwszego błędnego kroku**, brakującego sygnału oraz bariery, która powinna była zatrzymać problem.

## 45 Failure taxonomy

Przejdź przez kody. C1 oznacza niejasny kontrakt, C2 brak kontekstu, T1 problem narzędzia, M1 błąd rozumowania, V1 słabą weryfikację, O1 konflikt koordynacji.

Klasyfikacja zapobiega odruchowi „dodajmy więcej promptu”. Każda klasa wymaga innej poprawki.

## 46 Cost failure analysis

Powiedz, że agent może dostarczyć poprawny wynik zbyt wysokim kosztem. Szukamy tokenów, które nie zmieniły outcome, wielu odrzuconych prób i równoległości, która nie skróciła lead time.

Koszt koordynacji agentów jest realny. Więcej workerów nie gwarantuje szybszego wyniku.

## 47 Metrics

Omów sześć grup metryk. Outcome mierzy poprawność. Effort czas review i korekty. Flow przepływ pracy. Cost ekonomię. Safety naruszenia oraz blokady. Learning pokazuje, czy powtarzalne błędy zmieniają reguły zespołu.

Odrzuć liczbę wygenerowanych linii jako główny KPI. Więcej kodu może oznaczać gorsze rozwiązanie.

## 48 Hackathon

Otwórz ostatnią sekcję. Hackathon ma pozwolić działać szybko, ale w bezpiecznym pudełku. Wynikiem ma być dowód wartości i lista warunków dalszego wdrożenia, nie jedynie efektowne demo.

## 49 Hackathon contract

Omów trzy kolumny. Agent może czytać repo, edytować izolowany branch i uruchamiać lokalne testy. Sieć, nowe zależności i publikacja wymagają zgody. Produkcja, sekrety, destrukcja i merge bez review pozostają poza zakresem.

Poproś zespół, aby przed pracą dopasowały ten kontrakt do własnego problemu.

## 50

## Definition of done

Demo ma pokazać problem, poziom autonomii, guardrails, zweryfikowany wynik oraz koszt z porażkami. Nie wystarczy pokazać, że agent coś wygenerował.

Podkreśl, że uczciwy wynik negatywny ma wartość. Stwierdzenie „proces nie jest gotowy do autonomii” może ochronić firmę przed drogim i ryzykownym wdrożeniem.

## 51

## The first hour

Przejdź po sugerowanej godzinie. Najpierw problem i owner. Potem research repo. Następnie spec oraz guardrails. Dopiero później plan i podział plików. Pierwsza implementacja powinna być cienką, działającą ścieżką z testem.

Ten rytm ogranicza koszt fałszywego startu i szybko dostarcza informację zwrotną.

## 52

## Resources

Nie czytaj wszystkich linków. Wskaż trzy ścieżki: dokumentację Claude Code dla konfiguracji, „Building Effective AI Agents” dla architektury oraz „Demystifying Evals” dla mierzenia jakości.

Powiedz, że **SOURCES.md** zawiera pełną listę, w tym dokumentację **GitHub Spec Kit**, **GSD Core** i **BMad Method**, linki do RTK, Caveman i Ponytail oraz zastrzeżenia dotyczące szybko zmieniających się funkcji.

## 53

## Controlled autonomy

Zamknij pięcioma słowami: **kontrakt, kontekst, narzędzia, dowody, metryki**. Razem tworzą system, któremu można zaufać nie dlatego, że model brzmi pewnie, lecz dlatego, że potrafimy sprawdzić jego działanie.

Ostatnie zdanie możesz powiedzieć dosłownie: „Naszym celem nie jest maksymalna autonomia. Naszym celem jest największa autonomia, którą potrafimy bezpiecznie uzasadnić dowodami.”